

High-level Digital Design

Lab Assignment 6

Introduction to Catapult

0. Goals

In the classic, HDL-based, digital design, after the architecture is defined, the design team hand-codes the architectural decisions in the form of a Verilog or VHDL RTL description. Once coded, the hand-coded RTL design is tested, bugs are reported, and time is spent trying to hunt them down and fix them individually. Furthermore, modifications in the architecture, represent manual change in the HDL code, slowing down the development cycle. This methodology becomes impractical in today's extremely complex designs.

The purpose of High-Level Synthesis (HLS) is to generate production quality register transfer level (RTL) implementations from high-level specifications. HLS automates an otherwise manual process, eliminating the source of many design errors and accelerating a very long and iterative part of the development cycle.

The goal of this laboratory tutorial is to introduce the use of Catapult® Synthesis (Catapult), a high-level synthesis (HLS) tool that creates RTL implementations from compatible C, C++, and SystemC design specifications.

The C, C++, and SystemC design specification describes the structure and behavior of the design in such a way that Catapult can synthesize the interfaces, data structures, and loops to a specified ASIC or FPGA technology and produce an optimized RTL implementation.

Catapult produces an RTL implementation that is ready for simulation and gate-level synthesis and includes the following files:

- HDL files — For each target output language (VHDL, Verilog), Catapult creates a two types of HDL output: cycle and rtl.
- RTL simulation and synthesis scripts—Simulation and synthesis scripts for specified downstream tools such as Mentor Graphics QuestaSim or Synopsys DesignCompiler.
- Analysis reports— A cycle report and an RTL report.

In this lab tutorial, the following steps are presented:

- Synthesize a C++ design using scripts and Catapult GUI.
- Map an `ac_channel` variable to a hardware interface component (*Resource*).
- Analyze the design using the Catapult Gantt Chart and Design Analyzer to understand how the C++ has been synthesized to hardware.
- Run automatic verification of the C++ and RTL.

Catapult supports two libraries for specifying arbitrary bit-width signed, unsigned, and fixed-point numbers: the Algorithmic C (ac) library and the System C (sc) library. Although both of these libraries generally support similar functions, it's generally recommended to use the AC Datatype library due to the faster C simulation speeds and the seamless support for larger bit-width variables vs. smaller bit-width variables.

An example directory structure for organizing your C++ files along with Catapult project files should look something like:

|---Project directory

|----src

|----ccs

|----dat

|----sim

Where:

- "Project directory" is the current design directory
- "src" directory contains all C++ (*.cpp, *.cxx, *.C, *.h, *.hpp) source and header files and the Makefile. This is where the executable is compiled and linked.
- "ccs" contains the synthesis *.tcl scripts.
- "dat" directory contains any file I/O for the testbench.
- "sim" directory is for Matlab and Simulink projects and scripts.

1. Catapult tutorial

1. Copy from Atenea and extract *lab6.zip* on a folder. Name it, for instance, *lab6*.
2. Inside the *src* directory, there are three source files for this design: *mult_add_pipeline.cpp* is the synthesizable design. *tb.cpp* and *mult_add_pipeline_ref.cpp* are the part of the testbench. There are also two header files *mult_add_pipeline.h* and *mult_add_pipeline_ref.h* associated with their cpp counterparts.
3. Open the *mult_add_pipeline.cpp* file with a text editor and review the code. Observe that Algorithmic C data types are used. Bit-accurate *ac_channel* class is declared for inputs *a*, *b*, *c* and *output* result. For input *gain*, *ac_fixed* type is declared. *true* indicates signed value and *false* unsigned value. Finally, *bool* is the type of *gain_adjust* parameter. Internal variables store the *product* = *a* * *b* and the *sum* = *product* + *c* operations with input channels. Observe that channel objects are processed by means of *.read()* and *.write()* methods. In case *gain_adjust* is true, the final sum is multiplied by the gain: *sum* = *sum* * *gain*. The *result* variable is *sum* multiplied by 16 by means of a 4-bit shift to the left. Observe the applied castings and defined precisions to preserve data bits.
4. Compare *mult_add_pipeline.cpp* with the *mult_add_pipeline_ref.cpp*. The latter is the model with double precision used in this case as a comparison reference with the Algorithmic C version.
5. Observe also the *mult_add_pipeline.h* file, where the library declarations and function prototyping takes place.
6. Open a Linux terminal and *cd* to *lab6* directory.
7. Launch Catapult typing *catapult*. The Catapult GUI will open.
8. *File* → *New* → *Project...* Leave *Catapult* as project name, OK. If asked *Save current project first?* click *No*. The Catapult GUI at this stage is shown in Fig. 1. At the left, the Task Bar indicates the operations than can be selected. The

central window will display the design information during the process, and at the bottom there is the transcript for logs.

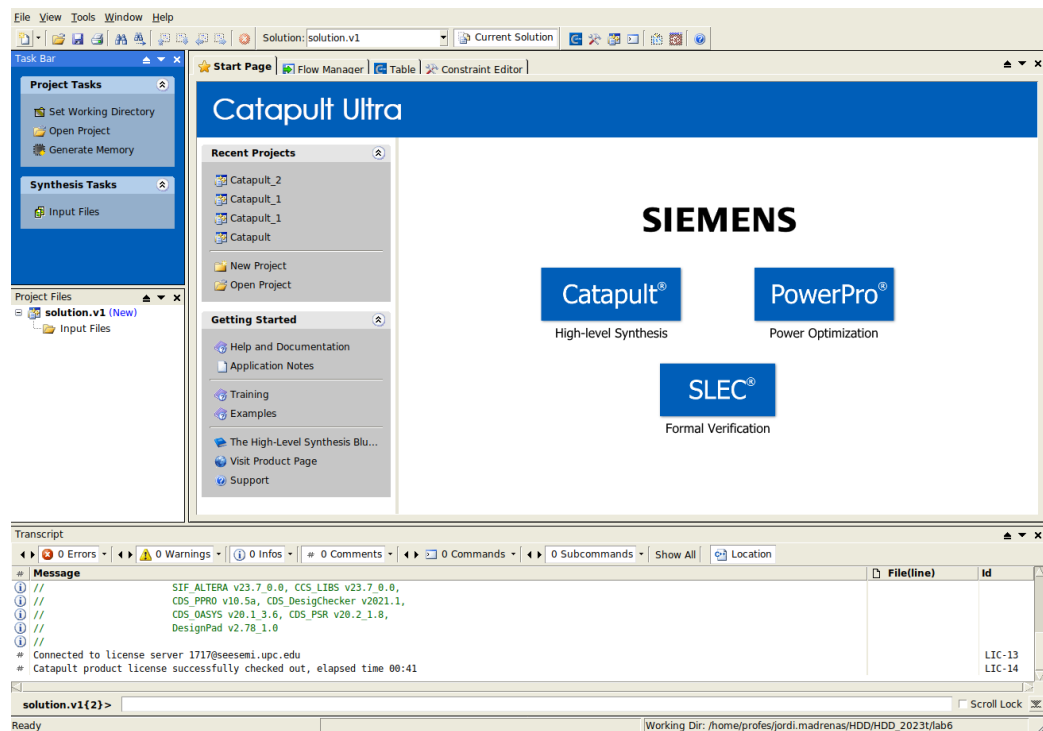


Figure 1. Catapult GUI.

9. In the *Task Bar* → *Synthesis Tasks* area, click *Input Files*.
10. Inside the *src* directory, select *mult_add_pipeline.cpp*, *mult_add_pipeline_ref.cpp*, *tb.cpp* with the mouse and CTL key. There is a column with the heading *Exclude* having a check box next to each file. This column is used to differentiate which files are part of the testbench, and conversely, which files are part of the design to be synthesized. Testbench files are excluded from being synthesized to RTL. Mark *exclude* box on *mult_add_pipeline_ref.cpp* and *tb.cpp* and *Open*.
11. Click on *Hierarchy* in the *Task Bar* → *Synthesis Tasks*. The source file is analyzed.
12. Observe that in the main window *mult_add_pipeline.cpp* has been set as the top level design.
13. Double-click on *mult_add_pipeline.cpp* to cross-probe on the C++ code.
14. To go back to the previous view, click on the *Constraint Editor* tab of the main window.
15. Click on *Libraries* in the *Task Bar*. The selected technology is displayed. If not yet, select *OasysRTL* as *RTL Synthesis Tool*, *Nangate* as *Vendor* and *045nm* as *Technology*. If not marked, below *Compatible Libraries* check *Catapult Base Lib for NandGateOpenCellLibrary typical*. Click *Apply*.
16. Click on *Mapping* in the *Task Bar*. This is where several constraints are set. For the moment, we will set the clock frequency. Below the *Module* section click on *core*, and set the frequency to 900 MHz. Click *Apply*.
17. Click on *Architecture* in the *Task Bar*. This opens the Architectural constraints dialog, where design constraints are applied.

- 17.1 Select *core* in the *Module* section. *Effort level (medium)* and *Design goal (area)*, among others, are selected here. Leave the defaults.
- 17.2 Select *main* folder in the *Module* section. Check *Pipeline* to enable hardware pipelining, a red arrow in the *main* icon appears.  Make sure *Stall Mode* is *Stall* and that *Loops can be Merged* is checked.
- 17.3 Expand the *interface* folder in the *Module* section and observe the input and output interfaces that have been created for the C++ design. Note that the bit widths are shown along with the port direction.
- 17.4 Expand the *a:rsc* interface and double click on the *a* variable to show it highlighted in the source editor.
- 17.5 Click on the *Constraint Editor* tab to go back to the previous view. Select now *a:rsc* and on the *Resource Type* area it has been mapped to a *ccs_in_wait* resource type, which is the default for *ac_channel* interfaces.
- 17.6 Click through the other resources under the interface folder. *a*, *b*, *c* and *result* are all mapped to **_wait* interfaces.
18. Click on *Resources* in the *Task Bar*. This will create the operations defined in the C++ description, as shown in the *Constraint Editor*: adder, multipliers, mux and read/write operators.
19. Click on *Schedule* in the *Task Bar*. This schedules the resources on the time diagram. The *Schedule* tab opens and, expanding *main*, the timing of the execution of each resource is shown. Observe the schedule of the product, sum, product and mux in separate clock cycles (Fig. 2).

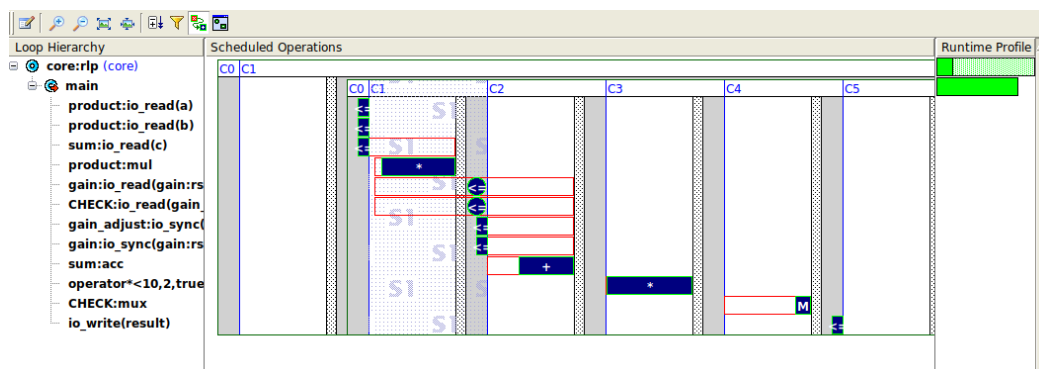


Figure 2. Scheduled operations.

20. Click on *RTL* in the *Task Bar*. This will map the operations on physical hardware. The *Constraint Editor* tab shows now the synthesized hierarchy.
21. On the main window, click the *Table* tab. In this design, a latency of 4 and a throughput of 1 is obtained. The explanation of the parameters is the following:
 - Latency Cycles: this is the number of clock cycles that, for this design implementation, are between an incoming sample and the corresponding outbound sample, i.e.: input-to-output cycles.
 - Latency Time: the amount of time corresponding to the latency cycles at the given clock rate. For our library, the unit is nanoseconds.
 - Throughput Cycles: this is the minimum number of cycles to complete the functionality of a single call of the synthesized function.
 - Throughput Time: this is the amount of time corresponding to the throughput cycles for the given clock rate.

- Slack: this is the minimum estimated slack time (i.e.: the portion of the clock cycle time that wasn't used for propagation through logic or wiring). A negative number would mean it is estimated the RTL design for this implementation will not meet timing.
 - Total Area: this is the estimated area, in units of the selected ASIC library, for this design implementation.
22. Back to *Schedule* in the *Task Bar*. If not yet, expand *main*. The *Scheduled Operations* diagram shows the task scheduling.
 - 22.1 Set the mouse over to see the area and delay. Note that the latter is drawn to scale. The white column of a C-step generally represents one clock cycle, which is 1.11 ns in the example. Gray areas represent transitions between clocks.
 - 22.2 Click on one operation, either in the text window or in the graphical window, to see the dependencies among operations.
 - 22.3 Double click an operation, either on the *Loop Hierarchy* or *Scheduled Operations* section, to cross-probe back to the C++ code.
 23. Double-click on the *Open Design Analyzer* in the *Project Files* section. A new interface, the Catapult *Design Analyzer*, is launched.
 24. Different views appear. In the order of the standard perspective: *Modules and Scopes*, that shows the module hierarchy and scope analysis, *Schematic*, that contains the RTL structure. *State*, that displays the objects for a given design status, *Input Files*, containing the C++ source code and *Schedule*, showing the operation scheduling (Fig. 3).

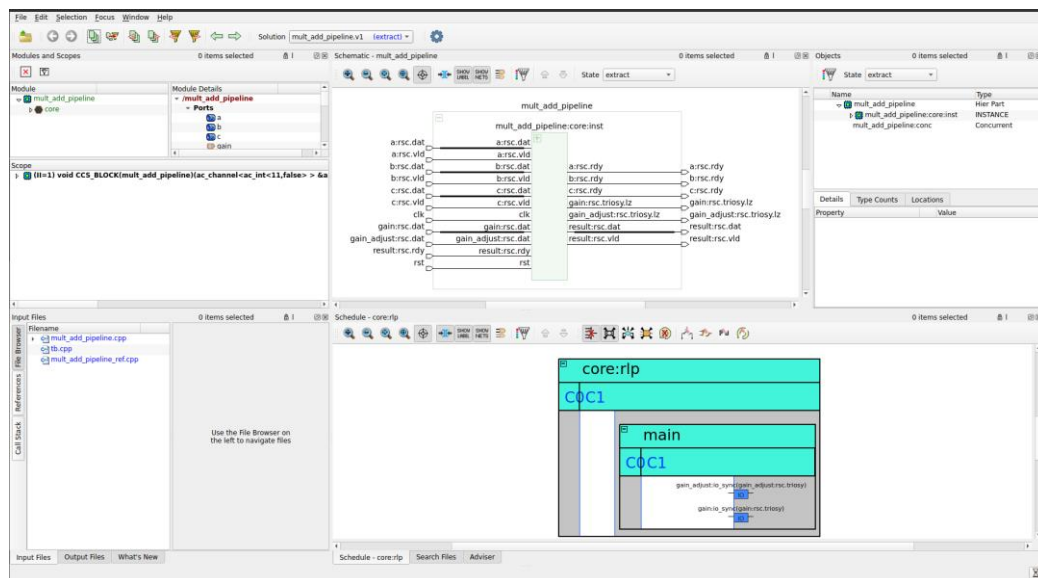


Figure 3. Catapult Design Analyzer GUI.

25. In the menu *Window* → *Perspectives...* click on *Reset Layout*. This resets the window layout to the default state. It is useful if some window disappears.
26. Click on *Selection* → *Filter root* to set the design filter at the top level of the hierarchy. It is equivalent to press the icon: 
27. Take a look at the top view of the *mult_add_pipeline* top level interface (Fig. 4).

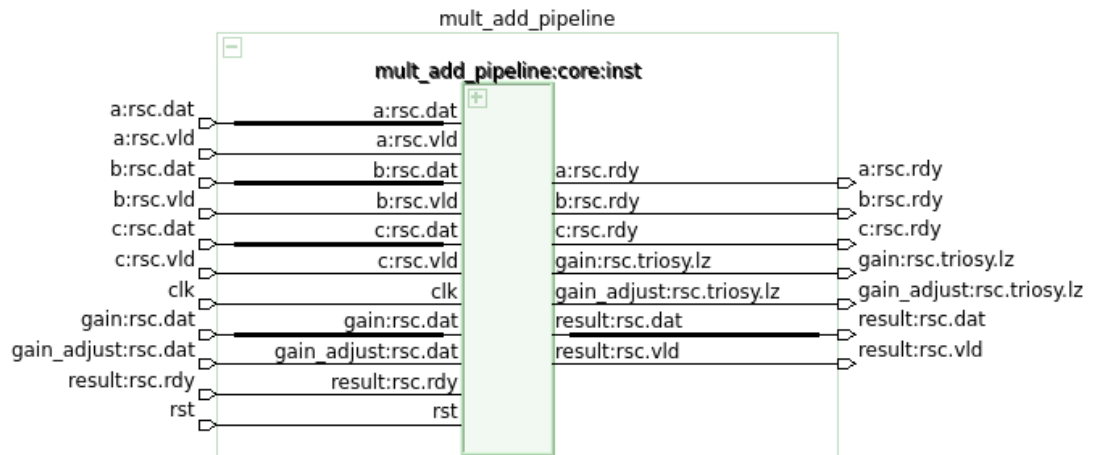


Figure 4. Top RTL Schematic view.

28. Expand it by clicking on the + sign at the top left of the green box. The design structure is shown, as well as the scheduled operations in the *Schedule* window (Fig. 5).
29. Inside the *Schedule* window, under the C1 column, click at the left red circle, corresponding to the first multiplier of the design. Note how the C++ code in the *Input Files* section is highlighted. The multiplier box in the *RTL Schematic* section is also highlighted. In the *Schedule* section, the lines (data dependencies) that cross the c-step boundary (gray area) imply registers that can be seen in the RTL schematic. It is better seen by clicking on the *Zoom Selected* icon  in the *Schematic* section. It is also easy to zoom dragging the left mouse button.
30. Click through the other operations in the *Schedule* view and note how they map back to the C++ and to the RTL Schematic.

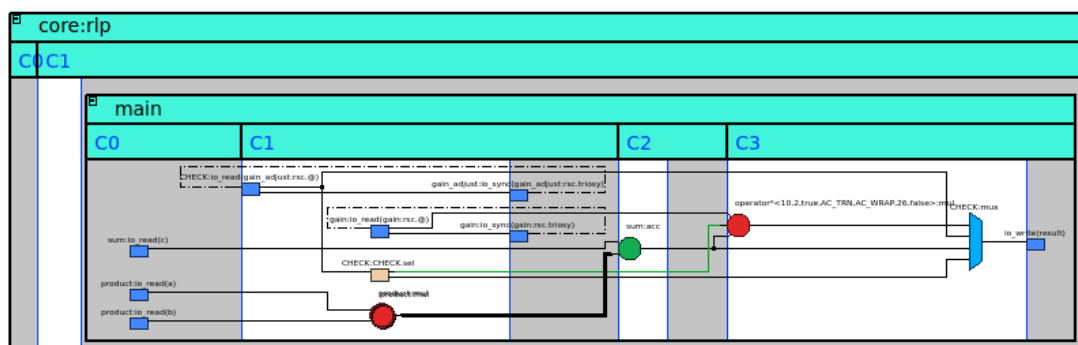


Figure 5. Schedule view.

31. Click on the blue small rectangles on the *Schedule* view. These are IO. Note how they map back to the C++.
32. On the *Objects* section (top right), in *State* field (at the top) select *allocate*. This is the default state for the *Objects* view, which matches the *Schedule* view of the design. All the operations are shown, but this does not account for resource sharing. Click on the *Type Counts* tab below. This gives a summary for all the objects for the current view and synthesis state. For instance, it can be seen that two multipliers are used in this example.
33. On the same *Objects* section, switch *State* from *allocate* to *extract* to view the resources of the actual hardware produced by the RTL generation. This includes registers and control logic after scheduling.

34. Go to *Window* in the main menu and select *Output Files*. This changes the bottom left window, but observe that *Input Files* can be accessed by means of the bottom tab.
35. In *Output Files*, expand *Verilog* and double-click on *rtl.v* to open the Verilog code of the RTL design in the subwindow.
36. Click through the *Schedule* and *RTL Schematic* views and note how the Verilog RTL code is highlighted. Note that not everything can be probed, as some things in the *Schedule* view may not exist in the RTL.
37. The *Modules and Scopes* section shows all the RTL instances added by synthesis such as IO, FSM, staller, etc. Make sure *extract* is selected in the *State* section.
- 37.1 In *Module Details*, expand *Generated* and double-click on the *a:rsci:inst* block, which is the interface for the *a* ac_channel (Fig. 6).

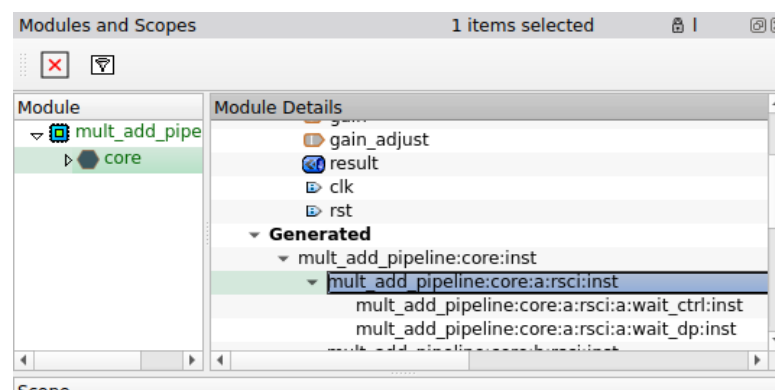


Figure 6. Modules and Scopes section.

- 37.2 Click the two sub-blocks under *a:rsci:inst*, to highlight the wait controller and FIFO, respectively.
- 37.3 In the *Schematic*, double-click on *wait_dp:inst* block, which is a 1-element FIFO, to push down into the hierarchy (Fig. 7). Select the top flip-flop (in purple color) and look at its details in the *Objects* view, *Details* tab. This 11-bit register is the 1-element FIFO of the *a* input interface.

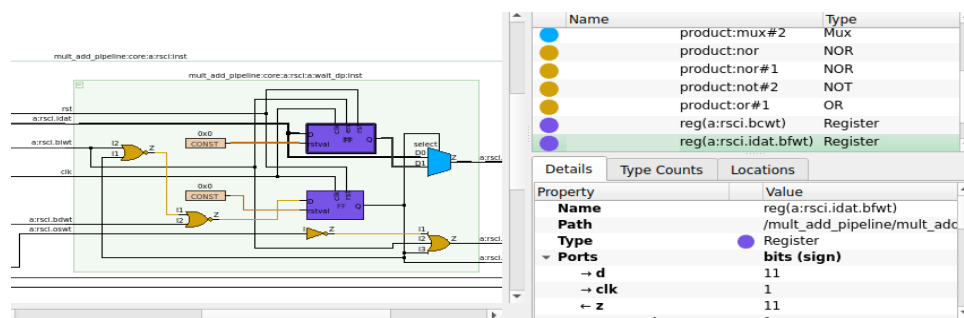


Figure 7. 11-bit register of the *a* input interface.

38. Other elements of the design can be checked following this procedure. When done, close the *Catapult Design Analyzer*.
39. Back to the main *Catapult Ultra Synthesis* window, expand the *Verification* folder in the *Project Files* section. Expand the *QuestaSIM* folder, as Questa is the simulator to be used in this lab. Then double-click on *Concat RTL Verilog output 'concat_sim_rtl.v' vs Untimed C++*. It calls the makefile to launch the

Verilog simulator. Questa Sim window opens, and the design is ready to execute the testbench verification.

40. Type *run -all* in the simulator *Transcript* section (bottom tab). The simulation is executed and the results should show report *PASSED* for 20 comparisons (Fig. 8).

```
# Checking results
# 'result'
#   capture count      = 20
#   comparison count   = 20
#   ignore count       = 0
#   error count        = 0
#   stuck in dut fifo  = 0
#   stuck in golden fifo = 0
#
# Info: scverify_top/user_tb: Simulation PASSED @ 661 ns
# ** Note: (vsim-6574) SystemC simulation stopped by user.
# 1
#
```

Figure 8. PASSED for 20 comparisons after testbench execution.

41. The time diagram can be seen in the *Wave* window. Click on the *Zoom Full* icon.
42. Zoom from 0 to 1500 ps by means of the icon: 
43. Observe the 4-cycle latency from *a_rsc_rdy* to *result_rsc_vld*, and the single-cycle throughput (Fig. 9).

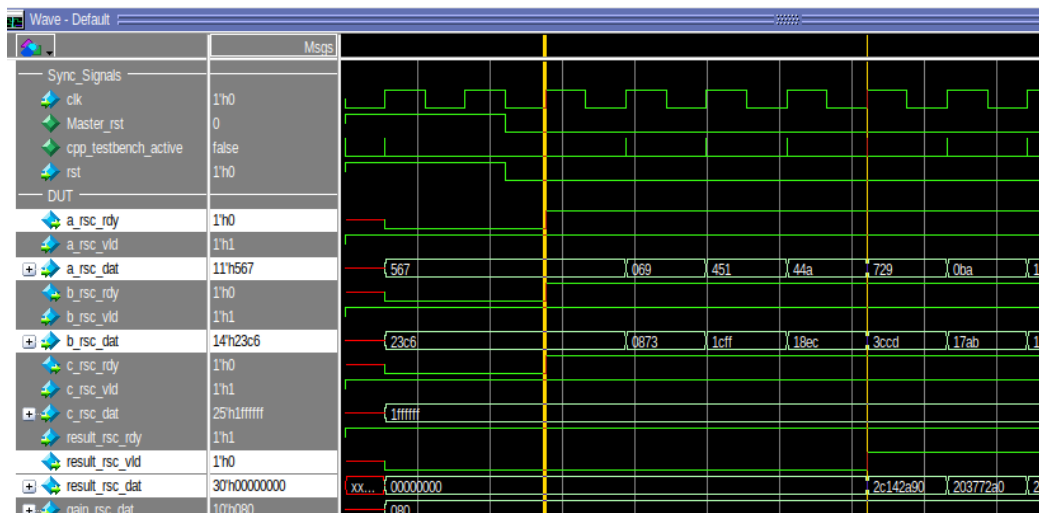


Figure 9. Time diagram of the simulation result.

44. Redo the previous steps for the Quartus – Altera – Cyclone V technology. Use as library only *BASE FPGA Library*. Try with a 200 MHz clock frequency. Skip steps 23 to 38.
45. By trial and error, find a clock frequency at which the slack is positive, and report the Table (latency, throughput, times, slack) data you obtained.